Security Update: Classic tokens have been revoked. Granular tokens are now limited to 90 days and require 2FA by default. Update your CI/CD workflows to avoid disruption. Learn more.

Pro Teams Pricing Documentation

npm

Sign Up Sign In

Search packages Search

pdf-parse

2.4.5 • Public • Published 4 months ago

Readme

Code Beta

2 Dependencies

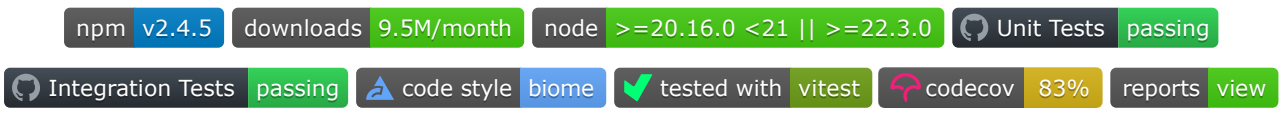
750 Dependents

53 Versions

pdf-parse

Pure TypeScript, cross-platform module for extracting text, images, and tables from PDFs.

Run 🤗 directly in your browser or in Node!



Getting Started with v2 (Coming from v1)

```
// v1
// const pdf = require('pdf-parse');
// pdf(buffer).then(result => console.log(result.text));

// v2
const { PDFParse } = require('pdf-parse');
// import { PDFParse } from 'pdf-parse';

async function run() {
        const parser = new PDFParse({ url: 'https://bitcoin.o

        const result = await parser.getText();
        console.log(result.text);
}

run();
```

Features

live demo

- CJS, ESM, Node.js, and browser support.
- Can be integrated with `React`, `Vue`, `Angular`, or any other web framework.
- **Command-line interface** for quick PDF processing: `CLI Documentation`
- `Security Policy`
- Retrieve headers and validate PDF: `getHeader`
- Extract document info: `getInfo`
- Extract page text: `getText`
- Render pages as PNG: `getScreenshot`
- Extract embedded images: `getImage`
- Detect and extract tabular data: `getTable`
- Well-covered with `unit tests`
- `Integration tests` to validate end-to-end behavior across environments.
- See LoadParameters and ParseParameters for all available options.
- Examples: `live demo`, `examples`, `tests` and `tests example` folders.

- Supports: `Next.js + Vercel`, Netlify, AWS Lambda, Cloudflare Workers.

Installation

```
npm install pdf-parse
# or
pnpm add pdf-parse
# or
yarn add pdf-parse
# or
bun add pdf-parse
```

CLI Installation

For command-line usage, install the package globally:

```
npm install -g pdf-parse
```

Or use it directly with npx:

```
npx pdf-parse --help
```

For detailed CLI documentation and usage examples, see: **CLI Documentation**

Usage

`getHeader` — Node Utility: PDF Header Retrieval and Validation

```
// Important: getHeader is available from the 'pdf-parse/node
import { getHeader } from 'pdf-parse/node';

// Retrieve HTTP headers and file size without downloading th
// Pass `true` to check PDF magic bytes via range request.
// Optionally validates PDFs by fetching the first 4 bytes (m
// Useful for checking file existence, size, and type before
// Node only, will not work in browser environments.
```

```
const result = await getHeader('https://bitcoin.org/bitcoin.p

console.log(`Status: ${result.status}`);
console.log(`Content-Length: ${result.size}`);
console.log(`Is PDF: ${result.isPdf}`);
console.log(`Headers:`, result.headers);
```

`getInfo` — Extract Metadata and Document Information

```
import { readFile } from 'node:fs/promises';
import { PDFParse } from 'pdf-parse';

const link = 'https://mehmet-kozan.github.io/pdf-parse/pdf/cl
// const buffer = await readFile('reports/pdf/climate.pdf');
// const parser = new PDFParse({ data: buffer });

const parser = new PDFParse({ url: link });
const result = await parser.getInfo({ parsePageInfo: true });
await parser.destroy();

console.log(`Total pages: ${result.total}`);
console.log(`Title: ${result.info?.Title}`);
console.log(`Author: ${result.info?.Author}`);
console.log(`Creator: ${result.info?.Creator}`);
console.log(`Producer: ${result.info?.Producer}`);

// Access parsed date information
const dates = result.getDateNode();
console.log(`Creation Date: ${dates.CreationDate}`);
console.log(`Modification Date: ${dates.ModDate}`);

// Links, pageLabel, width, height (when `parsePageInfo` is t
console.log('Per-page information:');
console.log(JSON.stringify(result.pages, null, 2));
```

`getText` — Extract Text

```
import { PDFParse } from 'pdf-parse';

const parser = new PDFParse({ url: 'https://bitcoin.org/bitco
const result = await parser.getText();
// to extract text from page 3 only:
// const result = await parser.getText({ partial: [3] });
await parser.destroy();
console.log(result.text);
```

For a complete list of configuration options, see:

- LoadParameters
- ParseParameters

Usage Examples:

- Parse password protected PDF: `password.test.ts`
- Parse only specific pages: `specific-pages.test.ts`
- Parse embedded hyperlinks: `hyperlink.test.ts`
- Set verbosity level: `password.test.ts`
- Load PDF from URL: `url.test.ts`
- Load PDF from base64 data: `base64.test.ts`
- Loading large files (> 5 MB): `large-file.test.ts`

`getScreenshot` — Render Pages as PNG

```
import { readFile, writeFile } from 'node:fs/promises';
import { PDFParse } from 'pdf-parse';

const link = 'https://bitcoin.org/bitcoin.pdf';
// const buffer = await readFile('reports/pdf/bitcoin.pdf');
// const parser = new PDFParse({ data: buffer });

const parser = new PDFParse({ url: link });
```

```
// scale:1 for original page size.
// scale:1.5 50% bigger.
const result = await parser.getScreenshot({ scale: 1.5 });

await parser.destroy();
await writeFile('bitcoin.png', result.pages[0].data);
```

Usage Examples:

- Limit output resolution or specific pages using ParseParameters
- `getScreenshot({scale:1.5})` — Increase rendering scale (higher DPI / larger image)
- `getScreenshot({desiredWidth:1024})` — Request a target width in pixels; height scales to keep aspect ratio
- `imageDataUrl` (default: `true`) — include base64 data URL string in the result.
- `imageBuffer` (default: `true`) — include a binary buffer for each image.
- Select specific pages with `partial` (e.g. `getScreenshot({ partial: [1,3] })`)
- `partial` overrides `first` / `last`.
- Use `first` to render the first N pages (e.g. `getScreenshot({ first: 3 })`).
- Use `last` to render the last N pages (e.g. `getScreenshot({ last: 2 })`).
- When both `first` and `last` are provided they form an inclusive range (`first..last`).

`getImage` — Extract Embedded Images

```
import { readFile, writeFile } from 'node:fs/promises';
import { PDFParse } from 'pdf-parse';

const link = new URL('https://mehmet-kozan.github.io/pdf-pars
// const buffer = await readFile('reports/pdf/image-test.pdf'
// const parser = new PDFParse({ data: buffer });

const parser = new PDFParse({ url: link });
const result = await parser.getImage();
await parser.destroy();
```

```
await writeFile('adobe.png', result.pages[0].images[0].data);
```

Usage Examples:

- Exclude images with width or height <= 50 px: `getImage({ imageThreshold: 50 })`
- Default `imageThreshold` is `80` (pixels)
- Useful for excluding tiny decorative or tracking images.
- To disable size-based filtering and include all images, set `imageThreshold: 0`.
- `imageDataUrl` (default: `true`) — include base64 data URL string in the result.
- `imageBuffer` (default: `true`) — include a binary buffer for each image.
- Extract images from specific pages: `getImage({ partial: [2,4] })`

`getTable` — Extract Tabular Data

```
import { readFile } from 'node:fs/promises';
import { PDFParse } from 'pdf-parse';

const link = new URL('https://mehmet-kozan.github.io/pdf-pars
// const buffer = await readFile('reports/pdf/simple-table.pd
// const parser = new PDFParse({ data: buffer });

const parser = new PDFParse({ url: link });
const result = await parser.getTable();
await parser.destroy();

// Pretty-print each row of the first table
for (const row of result.pages[0].tables[0]) {
        console.log(JSON.stringify(row));
}
```

Exception Handling & Type Usage

```
import type { LoadParameters, ParseParameters, TextResult } f
import { PasswordException, PDFParse, VerbosityLevel } from '

const loadParams: LoadParameters = {
        url: 'https://mehmet-kozan.github.io/pdf-parse/pdf/pa
        verbosity: VerbosityLevel.WARNINGS,
        password: 'abcdef',
};

const parseParams: ParseParameters = {
        first: 1,
};

// Initialize the parser class without executing any code yet
const parser = new PDFParse(loadParams);

function handleResult(result: TextResult) {
        console.log(result.text);
}

try {
        const result = await parser.getText(parseParams);
        handleResult(result);
} catch (error) {
        // InvalidPDFException
        // PasswordException
        // FormatError
        // ResponseException
        // AbortException
        // UnknownErrorException
        if (error instanceof PasswordException) {
                console.error('Password must be 123456\n', er
        } else {
                throw error;
        }
} finally {
```

```
        // Always call destroy() to free memory
        await parser.destroy();
}
```

Web / Browser

- Can be integrated into `React`, `Vue`, `Angular`, or any other web framework.
- **Live Demo:** `https://mehmet-kozan.github.io/pdf-parse/`
- **Demo Source:** `reports/demo`
- **ES Module**: `pdf-parse.es.js` **UMD/Global**: `pdf-parse.umd.js`
- For browser build, set the `web worker` explicitly.

CDN Usage

```
<!-- ES Module -->
<script type="module">

  import {PDFParse} from 'https://cdn.jsdelivr.net/npm/pdf-pa
  //// Available Worker Files
  // pdf.worker.mjs
  // pdf.worker.min.mjs
  // If you use a custom build or host pdf.worker.mjs yoursel
  PDFParse.setWorker('https://cdn.jsdelivr.net/npm/pdf-parse@

  const parser = new PDFParse({url:'https://mehmet-kozan.gith
  const result = await parser.getText();

  console.log(result.text)
</script>
```

CDN Options: https://www.jsdelivr.com/package/npm/pdf-parse

- `https://cdn.jsdelivr.net/npm/pdf-parse@latest/dist/pdf-parse/web/pdf-parse.es.js`

- `https://cdn.jsdelivr.net/npm/pdf-parse@2.4.5/dist/pdf-parse/web/pdf-parse.es.js`
- `https://cdn.jsdelivr.net/npm/pdf-parse@latest/dist/pdf-parse/web/pdf-parse.umd.js`
- `https://cdn.jsdelivr.net/npm/pdf-parse@2.4.5/dist/pdf-parse/web/pdf-parse.umd.js`

Worker Options:

- `https://cdn.jsdelivr.net/npm/pdf-parse@latest/dist/pdf-parse/web/pdf.worker.mjs`
- `https://cdn.jsdelivr.net/npm/pdf-parse@latest/dist/pdf-parse/web/pdf.worker.min.mjs`

Similar Packages

- pdf2json — Buggy, memory leaks, uncatchable errors in some PDF files.
- pdfdataextract — `pdf-parse`-based
- unpdf — `pdf-parse`-based
- pdf-extract — Non-cross-platform, depends on xpdf
- j-pdfjson — Fork of pdf2json
- pdfreader — Uses pdf2json
- pdf-extract — Non-cross-platform, depends on xpdf

> **Benchmark Note:** The benchmark currently runs only against `pdf2json`. I don't know the current state of `pdf2json` — the original reason for creating `pdf-parse` was to work around stability issues with `pdf2json`. I deliberately did not include `pdf-parse` or other `pdf.js`-based packages in the benchmark because dependencies conflict. If you have recommendations for additional packages to include, please open an issue, see `benchmark results`.

Supported Node.js Versions(20.x, 22.x, 23.x, 24.x)

- Supported: Node.js 20 (>= 20.16.0), Node.js 22 (>= 22.3.0), Node.js 23 (>= 23.0.0), and Node.js 24 (>= 24.0.0).
- Not supported: Node.js 21.x, and Node.js 19.x and earlier.

Integration tests run on Node.js 20–24, see `test_integration.yml`.

Unsupported Node.js Versions (18.x, 19.x, 21.x)

Requires additional setup see **docs/troubleshooting.md**.

Worker Configuration & Troubleshooting

See **docs/troubleshooting.md** for detailed troubleshooting steps and worker configuration for Node.js and serverless environments.

- Worker setup for Node.js, Next.js, Vercel, AWS Lambda, Netlify, Cloudflare Workers.
- Common error messages and solutions.
- Manual worker configuration for custom builds and Electron/NW.js.
- Node.js version compatibility.

If you encounter issues, please refer to the **Troubleshooting Guide**.

Contributing

When opening an issue, please attach the relevant PDF file if possible. Providing the file will help us reproduce and resolve your issue more efficiently. For detailed guidelines on how to contribute, report bugs, or submit pull requests, see: `contributing to pdf-parse`

Keywords

pdf **pdf-parser** **pdf-parse** **pdf.js** **pdfjs** **pdfjs-dist** **pdf2text** **pdf2json** **pdf2image** **pdf2pic** **pdf-to-text** **pdf-to-image** **pdf-viewer** **pdf-table** **pdf-tools** **pdf-utils** **pdf-screenshot** **pdf-thumbnail**

Provenance

Built and signed on
GitHub Actions
View build summary

Source Commit
github.com/mehmet-kozan/pdf-parse@e8e800b

Build File
.github/workflows/publish_npm_package.yml

Public Ledger
Transparency log entry

Share feedback

Install

```
npm i pdf-parse
```

Repository

github.com/mehmet-kozan/pdf-parse

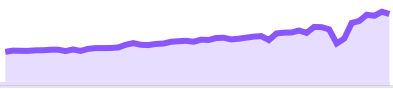
Homepage

mehmet-kozan.github.io/pdf-parse/

❤Fund this package

Weekly Downloads

2,308,987



Version

2.4.5

License

Apache-2.0

Unpacked Size

21.3 MB

Total Files

110

Last publish

4 months ago

Collaborators

Analyze security with Socket

Check bundle size

View package health

Explore dependencies

Report malware

Support

Help

Advisories

Status

Contact npm

Company

About

Blog

Press

Terms & Policies

Policies

Terms of Use

Code of Conduct

Privacy